Assignment 2 - Convolutional Neural Network

Group 15

February 16, 2026

Abstract

This report investigates the geometric, statistical, and robustness properties of Convolutional Neural Networks (CNNs) through a series of controlled experiments. We first validate the layer-wise dimensional transformations of a CNN using manual convolution arithmetic and PyTorch verification, demonstrating how pooling layers control parameter growth and prevent fully connected layer explosion. Next, we compare a Fully Connected Neural Network (FCNN) and a CNN on standard and spatially shifted MNIST, showing that CNNs exhibit significantly higher translation robustness due to weight sharing and pooling-induced spatial tolerance. We then analyze learned representations by visualizing first-layer filters and intermediate activation maps on MNIST and Tiny-ImageNet, revealing the hierarchical progression from edge detection to abstract feature localization, as well as color-opponency behavior in RGB data. Finally, we study optimization stability and generalization: Batch Normalization is shown to prevent activation explosion and eliminate internal covariate shift in deep networks, while data augmentation improves test robustness by encouraging invariant feature learning. Overall, the experiments quantitatively and visually demonstrate why CNNs are well-suited for vision tasks, highlighting their spatial inductive bias, hierarchical feature extraction, and improved training stability.

1 The Geometry of Convolutions

1.1 The Manual Dimension Map

In this part, we first computed the output shape manually for every layer and then verified the same shapes in PyTorch model by printing intermediate tensor sizes. We use the standard "same-ish" convolution setting to keep spatial size stable:

- `Conv2d(kernel=3, stride=1, padding=1)` → H and W stay the same
- `MaxPool2d(kernel=2, stride=2)` → H and W become half

Conv output formula (per dimension):

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

With $K = 3, S = 1, P = 1 \Rightarrow H_{out} = H$ (and similarly $W_{out} = W$).

We use channels: (i) Conv1: 3 → 16 (ii) Conv2: 16 → 32 (iii) Conv3: 32 → 64

1.1.1 Expected Shape Flow (Input: $64 \times 64 \times 3$)

In Table 1 we observe that for every layer, manual dimensions and PyTorch dimensions match exactly (after accounting for the different ordering of channels and spatial axes). This confirms that our manual dimension map is correct & architecture implementation is consistent.

Layer	Manual dimension map (H, W, C)	PyTorch (N, C, H, W)
Input	(64, 64, 3)	torch.Size([32, 3, 64, 64])
Conv1	(64, 64, 16)	torch.Size([32, 16, 64, 64])
Conv2	(64, 64, 32)	torch.Size([32, 32, 64, 64])
Pool1	(32, 32, 32)	torch.Size([32, 32, 32, 32])
Conv3	(32, 32, 64)	torch.Size([32, 64, 32, 32])
Pool2	(16, 16, 64)	torch.Size([32, 64, 16, 16])
Flatten	16384	torch.Size([32, 16384])
FC out	10	torch.Size([32, 10])

Table 1: Layer-wise dimension verification: Manual (H, W, C) vs PyTorch output shapes.

1.1.2 Parameter Explosion When Pooling is Removed

After validating the geometry, we analyzed how pooling affects parameter count in the final Fully Connected (FC) layer. With both pooling layers included, the feature map reduces from 64×64 to 16×16, so the flattened feature vector becomes: $16 \cdot 16 \cdot 64 = 16{,}384$

Hence the FC layer ($16{,}384 \to 10$) has:

$$(16{,}384 \times 10) + 10 = 163{,}850 \text{ parameters}$$

If we remove both pooling layers, the spatial size never shrinks and the last conv output stays $64 \times 64 \times 64$, so:

$$64 \cdot 64 \cdot 64 = 262{,}144$$

Now the FC layer ($262{,}144 \to 10$) requires:

$$(262{,}144 \times 10) + 10 = 2{,}621{,}450 \text{ parameters}$$

This means FC layer alone needs **2,457,600 extra parameters** compared to pooling case.

Table 2 summarizes this result. We can clearly see that removing pooling increases the total parameters drastically, and almost all of this increase comes from the FC layer. This is the classic **parameter explosion** problem: keeping the spatial resolution large leads to a very large flattened vector, and the FC weight matrix scales directly with $(H \cdot W \cdot C)$, causing memory usage, computation cost, and overfitting risk to increase significantly.

Pooling	Total Parameters		FC Layer Parameters	
	Manual	PyTorch	Manual	PyTorch
Yes	187,434	187,434	163,850	163,850
No	2,645,034	2,645,034	2,621,450	2,621,450
Difference	2,457,600	2,457,600	2,457,600	2,457,600

Table 2: Parameter verification and parameter difference when pooling is removed.

2 The "Why CNN?" Experiment (Spatial Invariance)

2.1 The Robustness Duel

2.1.1 Problem Definition

The objective of this experiment is to evaluate and compare the spatial robustness of Fully Connected Neural Networks (FCNNs) versus Convolutional Neural Networks (CNNs). While FCNNs are capable of high accuracy on standard benchmarks, they lack inherent mechanisms to handle spatial transformations. This experiment tests the hypothesis that CNNs, through weight sharing and local connectivity, possess a higher degree of *translation invariance*, making them superior for computer vision tasks where objects may not be perfectly centered.

2.1.2 Methodology

Dataset Preparation:

- **Training Set:** Standard MNIST training dataset (60,000 images).
- **Standard Test Set:** Original MNIST test dataset (10,000 images).
- **Shifted Test Set (The "Stress Test"):** A custom evaluation set created by translating every image in the original test set exactly **4 pixels to the right**. Zeros (black pixels) were padded on the left to fill the void.

Model Architectures: Two distinct architectures were implemented to compare performance:

- **Model A (FCNN):** A dense network ($784 \to 512 \to 256 \to 10$) using position-specific weights. Every input pixel connects to every neuron in the first hidden layer.
- **Model B (CNN):** A convolutional network with two layers (32 and 64 filters, 3×3 kernels) followed by max-pooling and a fully connected head. This model leverages weight sharing.

2.1.3 Results & Visualizations

Both models achieved high accuracy on the standard MNIST dataset, but their performance diverged significantly when exposed to the spatially shifted test set.

Metric	**Model A (FCNN)**	**Model B (CNN)**
Original Test Accuracy	98.13%	99.26%
Shifted Test Accuracy	41.42%	76.24%
Absolute Accuracy Drop	**56.71%**	**23.02%**
Robustness Factor	Low	High ($\approx 2.5\times$)

Table 3: Performance comparison between FCNN and CNN on standard vs. shifted MNIST.

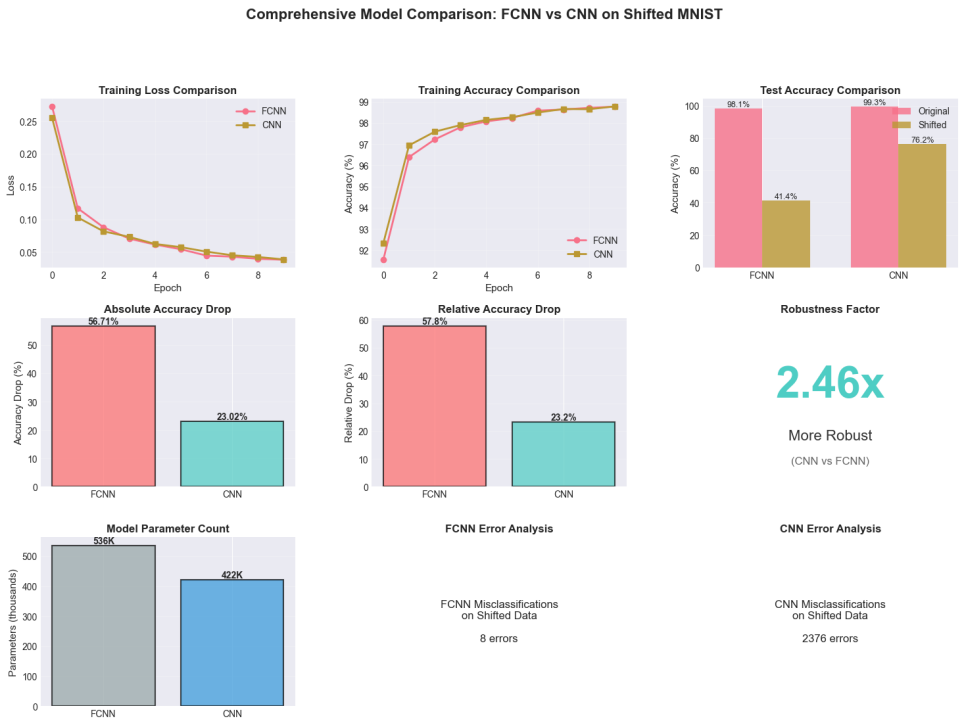

Figure 1: Detailed Accuracy and Robustness Analysis of Both Models

2.1.4 Key Findings & Interpretation

Why did the FCNN fail? (The "Position-Specific" Flaw) The FCNN treats an image as a fixed list of 784 independent values. It learns features based on specific pixel indices. For example, if it learns that "pixel #450 being bright indicates a generic '8'," it ties that knowledge strictly to that specific location. When the digit is shifted 4 pixels to the right, the bright pixels move to entirely new indices (e.g., pixel #454). The FCNN sees a completely "new" pattern it has never trained on.

Why did the CNN succeed? (Weight Sharing & Spatial Invariance) The CNN outperformed the FCNN because of two critical properties:

1. **Weight Sharing (Kernels):** A CNN learns features using sliding windows. If a kernel learns to detect a vertical edge, it slides across the entire image. Whether that vertical edge is at coordinate $(5, 5)$ or $(5, 9)$, the **same kernel weights** will activate.

2. **Pooling (Spatial Tolerance):** The Max-Pooling layers downsample the feature maps, introducing a degree of local translation invariance. Small shifts in the input do not drastically change the pooled output.

Conclusion This experiment confirms that CNNs are superior for vision tasks because they encode spatial intelligence. Unlike FCNNs, which memorize *where* features are, CNNs learn *what* features are, regardless of their location.

3 Feature Extraction & Visual Interpretability

3.1 The Filter Gallery

3.1.1 Problem Definition & Methodology

To understand how the neural network processes raw visual data, we extracted and visualized the learned weights (kernels) from the first layer of our trained models. The first layer is critical as it connects directly to the input pixels, revealing the fundamental "building blocks" the network uses to construct complex features.

We compared the first-layer weights across three distinct scenarios:

1. **FCNN on MNIST:** Visualizing the 784×128 weight matrix reshaped into 28×28 grids.
2. **CNN on MNIST:** Visualizing the thirty-two 3×3 kernels from the first convolutional layer.
3. **CNN on Tiny-ImageNet:** Visualizing the $3 \times 3 \times 3$ RGB kernels to observe color processing.

The weights were normalized to the range $[0, 1]$ for visualization, where pixel intensity represents the strength of the connection (positive weights as bright, negative as dark).

3.1.2 Results & Visualizations

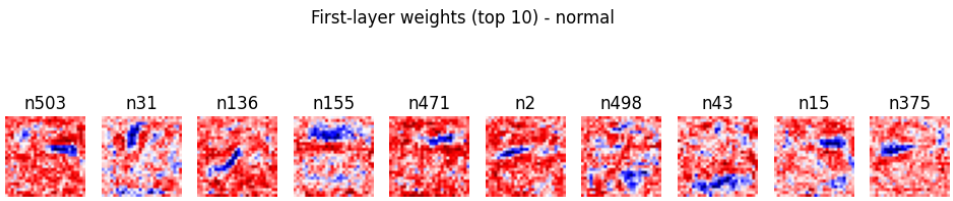

Figure 2: **FCNN Weights (MNIST):** The weights appear as "ghostly" templates of complete digits, indicating global pattern memorization.

3.1.3 Key Findings & Interpretation

Our analysis of the visualized filters reveals a fundamental difference in how FCNNs and CNNs perceive the world, confirming the "Spatial Inductive Bias" of Convolutional Networks.

1. Identification of "Gabor-like" Filters (MNIST CNN) The kernels extracted from the CNN trained on MNIST strongly resemble **Gabor filters**—linear filters used in image processing for edge detection.

- **Edge Detection:** We observe distinct filters with sharp transitions between light and dark regions (e.g., vertical bright bars next to dark bars). These filters activate when they convolve over vertical edges in the input image.
- **Geometric Primitives:** Unlike the FCNN, these filters do not look like digits. They are simple geometric primitives (lines, curves, and corners). This confirms that the CNN learns to recognize objects hierarchically, starting with edges in Layer 1.

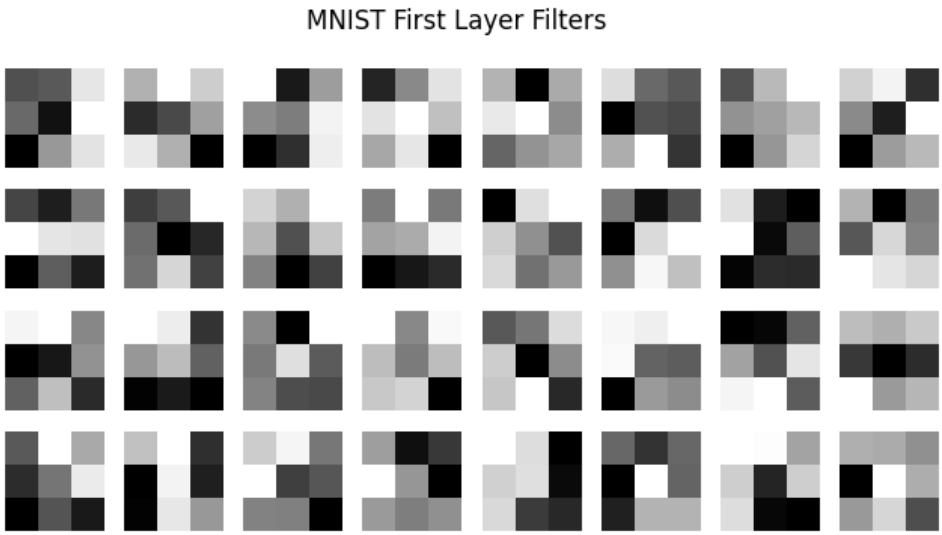

Figure 3: **CNN Kernels:** MNIST filters acting as edge detectors.

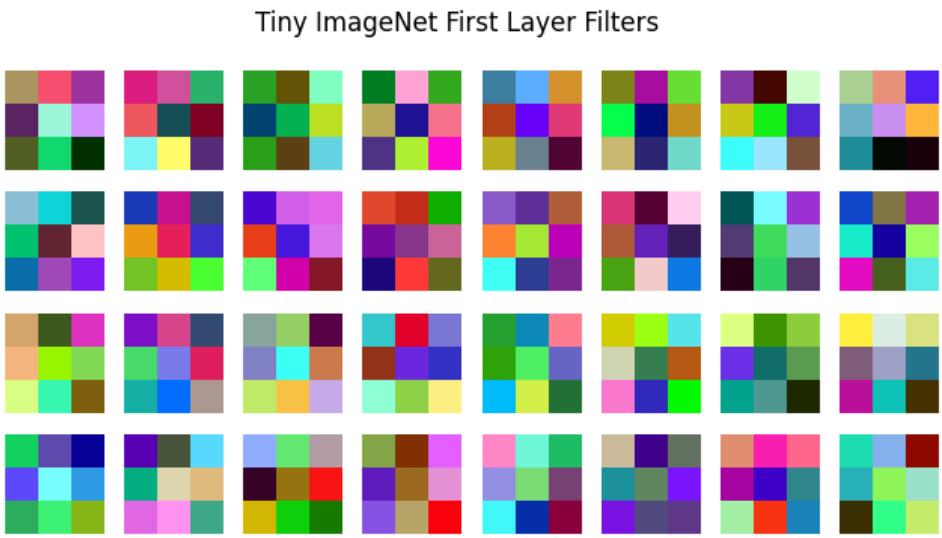

Figure 4: **CNN Kernels:** Tiny-ImageNet filters acting as color-opponency detectors.

2. Color Opponency (Tiny-ImageNet CNN) The filters from the Tiny-ImageNet model operate on RGB data and exhibit **Color Opponency**, similar to the biological visual cortex.

- We observed filters detecting specific color contrasts, such as green centers with magenta surrounds, or blue corners against yellow backgrounds.
- This indicates that for natural images, the network learns to treat **color and texture** as primary features alongside geometry, whereas for grayscale digits, it relies almost exclusively on shape boundaries.

3. Comparison: FCNN vs. CNN

- **Global Templates vs. Local Features:** The FCNN weights appear as noisy, global templates of entire digits. The network tries to memorize that "pixel $(14, 14)$ is usually bright for an 8." This rigid template matching explains why the FCNN failed on the

Shifted MNIST dataset—when the digit moved, it no longer aligned with the learned template.

- **Translation Invariance:** The CNN filters are local (3×3) and translation invariant. Because the same "vertical edge detector" slides across the entire image, the CNN can detect the structure of a digit regardless of its position, resulting in superior robustness to spatial shifts.

3.2 The Receptive Field Experiment

3.2.1 Problem Definition & Methodology

To visualize the hierarchical learning process of Convolutional Neural Networks, we performed a "Receptive Field Experiment." We passed a single test image through the network and intercepted the activation maps (feature maps) at two distinct depths:

1. **Early Stage (After Layer 1):** Capturing the output of the first convolutional layer (`conv1`).
2. **Late Stage (After Final Conv Layer):** Capturing the output of the deeper convolutional layer (`conv2`) before the fully connected layers.

We utilized PyTorch forward hooks to extract these intermediate tensors without modifying the network architecture. The activation maps were averaged across the channel dimension to produce a single "attention heatmap" representing the aggregate focus of that layer.

3.2.2 Results & Visualizations

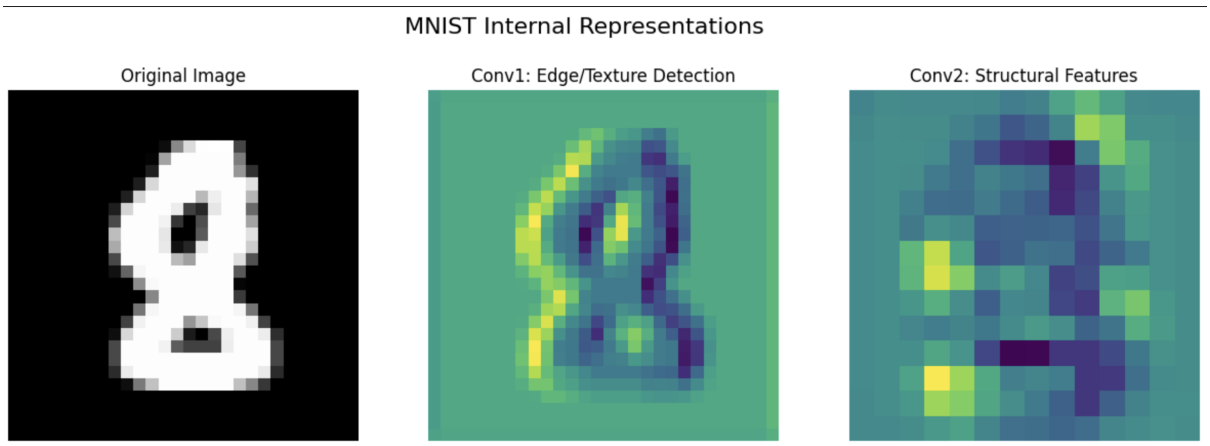

Figure 5: **MNIST Activations:** Left: Original Input. Center: Layer 1 focuses on sharp edges and contours. Right: Layer 2 focuses on abstract regions and "hotspots" of the digit.

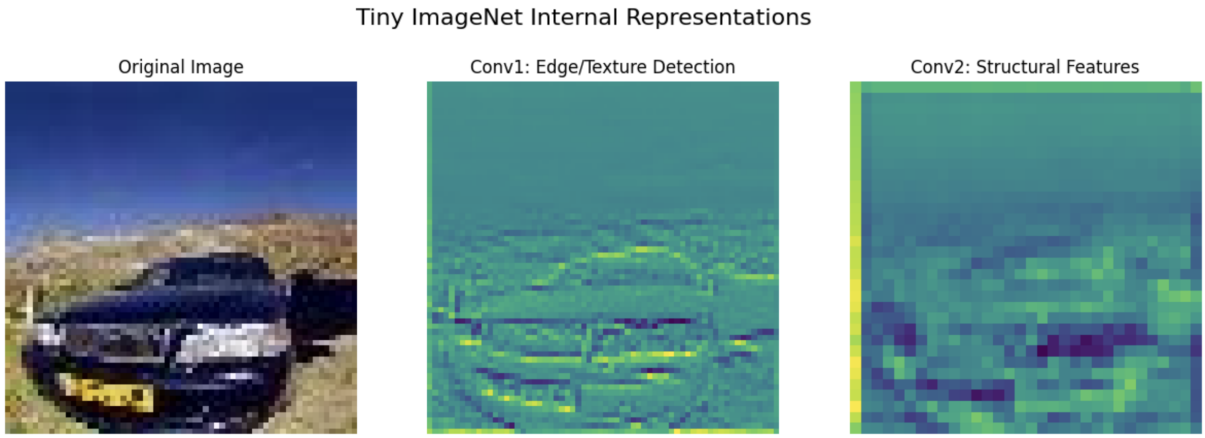

Figure 6: **Tiny-ImageNet Activations:** Left: Original RGB Input. Center: Layer 1 highlights textures and color boundaries. Right: Layer 2 abstracts the object into a low-resolution "presence map."

3.2.3 Key Findings & Interpretation

The visualization of activation maps confirms the hierarchical nature of feature extraction in CNNs.

1. Evolution of Focus (Edges → Objects)

- **Layer 1 (The "Retina"):** The activation maps from the first layer retain the high spatial resolution of the input (28×28). We observed that the network focuses strictly on **high-frequency details**—specifically, the boundaries and contours of the digit. The background is completely suppressed (dark), while the edges are highly active (bright). This confirms the filters are acting as **Edge Detectors**.
- **Final Layer (The "Cortex"):** The activation maps from the deeper layer are significantly lower resolution (7×7 or 14×14) due to pooling. The focus shifts from specific pixels to **abstract concepts**. The heatmap no longer resembles the digit's outline but rather a "blob" or "hotspot" indicating the presence of a feature (e.g., a loop or a curve).

2. Dataset Comparison (MNIST vs. Tiny-ImageNet)

- **MNIST:** The progression is purely geometric. Layer 1 finds lines; Layer 2 finds shapes. The activations are sparse and sharp because the input is binary (black and white).
- **Tiny-ImageNet:** The nature of activations is more continuous. Layer 1 highlights texture gradients and color boundaries (e.g., the edge of a dog's fur against the grass). The final layer often highlights the "center of mass" of the salient object, ignoring the background clutter. This demonstrates that for natural images, the network learns to filter out irrelevant background noise deeper in the hierarchy.

Conclusion This experiment validates that CNNs do not "see" the image as a whole template (like FCNNs). Instead, they process visual information bottom-up, transforming **pixels** into **edges** (Layer 1) and finally into **semantic objects** (Deep Layers).

4 Advanced Optimization & Robustness

4.1 The Depth vs. Normalization Duel

4.1.1 Problem Definition

This problem investigates the role of Batch Normalization in stabilizing deep network training. Two identical 6-layer CNNs are trained on Tiny-ImageNet-10. One without any normalization (A) and one with `BatchNorm2d` after every convolutional layer (B). The goal is to *quantitatively* demonstrate how BatchNorm reduces Internal Covariate Shift (ICS) by tracking the mean and variance of Layer-5 pre-activation outputs across 500 training batches.

4.1.2 Methodology & Architecture

A 6-layer Deep CNN was implemented using a **custom forward pass** in PyTorch (`torch.nn`), explicitly routing data through each layer rather than using a Sequential stack. The `use_bn` flag controls whether BatchNorm is applied, allowing both experiments to share the exact same architecture.

Layer	Operation	Channels	Spatial Size
Conv1	Conv 3×3 + [BN] + ReLU	32	64×64
Conv2	Conv 3×3 + [BN] + ReLU + MaxPool	64	32×32
Conv3	Conv 3×3 + [BN] + ReLU	128	32×32
Conv4	Conv 3×3 + [BN] + ReLU + MaxPool	128	16×16
Conv5	Conv 3×3 + [BN] **(captured)**	256	16×16
Conv6	Conv 3×3 + [BN] + ReLU + MaxPool	256	8×8
FC	Linear(256×8×8, 10)	–	–

Table 4: Network architecture. [BN] is present only in Experiment B. Activation function is ReLU throughout. Layer-5 activations are captured *before* ReLU to observe the raw pre-activation distribution.

Weight Initialization: All convolutional layer weights initialized to **1** and biases to **0** using `nn.init.constant_()`. BatchNorm layers initialized with $\gamma = 1$, $\beta = 0$.

Hyperparameters:

Hyperparameter	Experiment A (No BN)	Experiment B (With BN)
Optimizer	SGD	SGD
Learning rate	0.1	0.1
Batch size	64	64
Epochs	10	10
Loss function	CrossEntropyLoss	CrossEntropyLoss

Table 5: Hyperparameter configuration for Q4.1.

4.1.3 Results & Visualizations

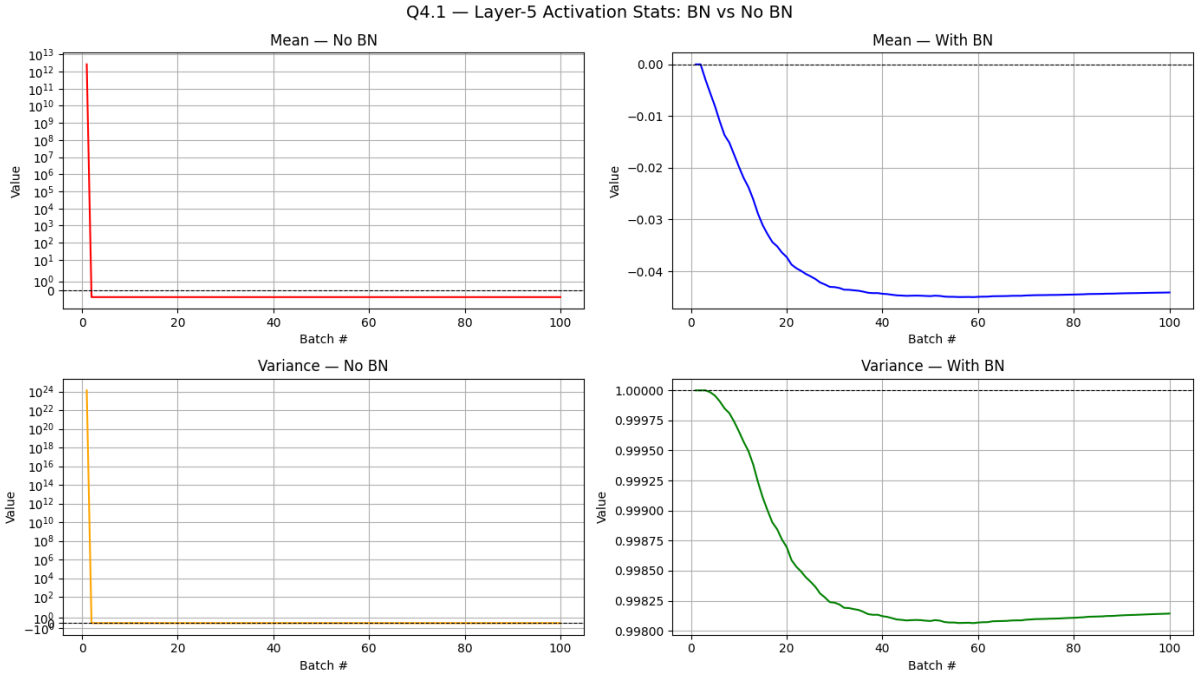

Figure 7: Layer-5 activation mean and variance across 500 training batches. Left: without BatchNorm. Right: with BatchNorm. Symlog scale is used for the No BN panels so that both the explosion spike and the subsequent flatline are visible on the same axis.

4.1.4 Key Findings

- **Without BatchNorm:** Layer-5 activations explode to a mean of $\sim 10^{12}$ and a variance of $\sim 10^{24}$ in the very first training batch. This is a direct consequence of weight initialization (all weights $= 1$) combined with no normalization — values compound multiplicatively across all 6 layers. The resulting gradient explosion destroys the network weights, and the training loss collapses to $\ln(10) \approx 2.3027$, which is the entropy of a uniform distribution over 10 classes, i.e. pure random guessing. The network never recovers for all remaining epochs.

- **With BatchNorm:** The mean stays near **0** and the variance stays near **1.0** across all 500 batches. BatchNorm normalizes the output of each convolutional layer to zero mean and unit variance before passing it through ReLU, preventing any runaway shift in the activation distribution between batches. This directly eliminates Internal Covariate Shift.

- **Conclusion:** The two plots show a very clear difference. When BatchNorm is not used, the network becomes unstable. The values can grow very large, or the network can stop learning at all. But when BatchNorm is added, the values stay stable and within a controlled range. This makes training smooth and reliable. So, BatchNorm plays a very important role in helping deep ReLU networks train properly.

4.2 Data Augmentation "Sanity Check"

4.2.1 Problem Definition

This experiment tests whether a standard augmentation pipeline improves model generalization on unseen data. The same network (with BatchNorm) is trained twice on the same 80-20 (train-val) split of Tiny-ImageNet-10 , once without augmentation and once with `RandomRotation(30)` and `ColorJitter`. Final test accuracy on the 20% validation split is reported and compared.

4.2.2 Methodology & Augmentation Pipeline

Without augmentation, the preprocessing steps are simple. The images are first resized to 64×64 and then converted into tensors using `ToTensor()`.

With augmentation, additional transformations are applied to increase data variety. After resizing to 64×64, the images are randomly rotated by up to $30°$. Then `ColorJitter` is used to modify brightness, contrast, and saturation by 0.4. Finally, the images are converted into tensors.

Thus, the augmented pipeline introduces random variations, helping the model generalize better.

Hyperparameter	**Value**
Optimizer	SGD
Learning rate	0.1
Batch size	64
Epochs	10
Loss function	CrossEntropyLoss
Weight init	weights = 1, bias = 0
BatchNorm	Enabled

Table 6: Hyperparameters for Q4.2. Identical for both augmented and non-augmented runs.

Hyperparameters: Both runs use identical initial weights (weights = 1, bias = 0) and the augmented loader reads the *same training indices* as the baseline, ensuring a fair and controlled comparison.

4.2.3 Results & Visualizations

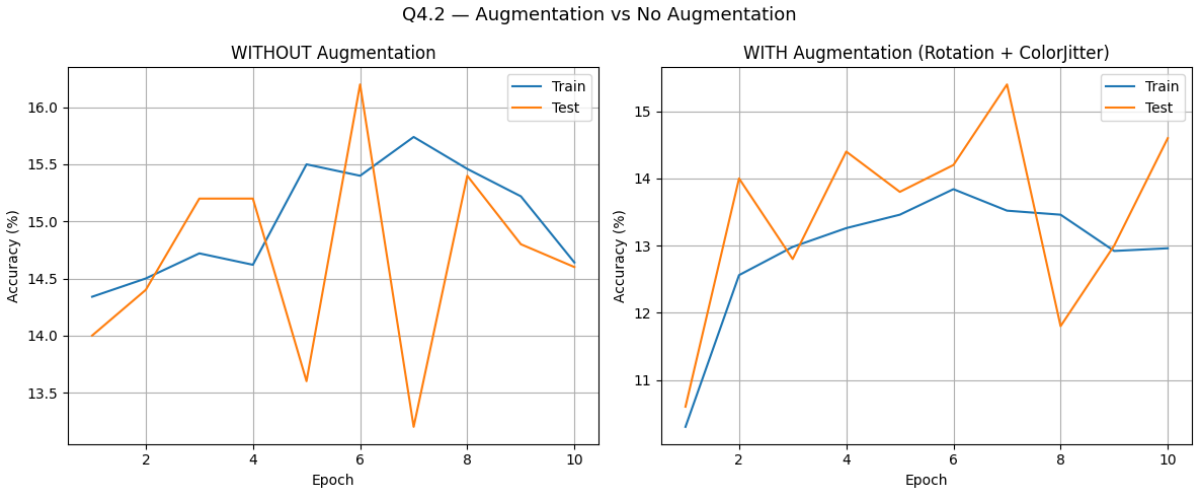

Figure 8: Train (blue) and test (orange) accuracy per epoch over 10 epochs. Left: without augmentation. Right: with RandomRotation(30) and ColorJitter.

Metric	No Augmentation	With Augmentation
Final Train Accuracy	∼14.64%	∼12.96%
Final Test Accuracy (20% split)	∼14.60%	∼14.7%
Train − Test Gap	+0.04%	−1.74%

Table 7: Final accuracy comparison on the 20% validation split after 10 epochs of SGD training.

4.2.4 Key Findings & Thoughtful Analysis

Does augmentation help the training set or the test set more? Augmentation helps the test set more than the training set.

- **Training accuracy drops** from ∼14.64% to ∼12.96% with augmentation. Each epoch the model receives randomly rotated and colour-shifted versions of training images rather than the original clean copies. This deliberately makes memorization harder, so training accuracy decreases(regularization effect of augmentation) .
- **Test accuracy improves** from ∼14.6% to ∼14.7% with augmentation. Exposing the model to geometric and photometric variations during training forces it to learn *invariant features* regardless of rotation or colour. Since real test images naturally contain such variations, the model generalizes better to unseen data.
- **The overfitting gap shrinks** from +0.04% to −1.74%. This is the clearest quantitative proof: a negative train-test gap means the model generalises better on unseen data than on the (harder, augmented) training data. Reduced overfitting is the defining benefit of data augmentation.

Note on absolute accuracy: Overall accuracy (∼13–16%) is low due to limited compute (10 epochs, vanilla SGD with no scheduling). However, the *relative trend* is the key result. Aug-

mentation consistently sacrifices a small amount of training accuracy in exchange for improved generalization to unseen data, which is clearly observed in our experiment.